



۸۱۰۱۰۱۴۹۲

سیستم‌های یادگیری ماشین توزیع‌شده
نام و نام خانوادگی: علیرضا کریمی



تمرین دستی ۲

۱

در واقع در روش *bagging*، می‌ایم مدل‌های مختلف با *data* های مختلف (*bootstrap* های متفاوت) *train* می‌کنیم. بعد روی نتیجه رای گیری می‌کنیم. طبق چیزی که سرچ کردم، *bootstrap* میشه یه زیر مجموعه از دیتا نه کل دیتا و ثانیاً میتونه شامل دیتا های تکراری هم باشه حالا که مدل‌های مختلف با *bootstrap* های مختلف *train* شدن، خروجی و نتیجه شون *aggregate* میشه به صورت رای اکثریت (در روش های *classification*) و یا ممکنه میانگین گیری از خروجی‌ها گرفته بشه (در روش های *regression*)

۲

چون *gpu* های قدیمی اتصال ۲ به ۲ رو خوب *support* می‌کردن به کمک *nvlink*، برای همین از توپولوژی *ring* استفاده میشه. ثانیاً مقیاس پذیر یعنی با اضافه شدن *gpu* فقط دو تا اتصال به بعدی و قبلی اضافه شده برعکس *fc* که باید به همه اتصال میداشت (اتصال *all - to - all*) و سوییچ مرکزی که *bottleneck* بشه نداریم هزینه اجرائش هم کمه (به واسطه همین تعداد لینک کمتر) عملیات های *ring allreduce* و مثلاً برودکست و اینا رو هم داریم و *support* میکنه که برای *train* کردن مدل‌های هوش مصنوعی به صورت *distributed* لازمه

۳

روش *sfb* اینطوریه که ماتریس رو به دو تا وکتور بشکنیم. و به جای پاس دادن کل پارامتر های ماتریس که میتونه از اردر $n * m$ باشه، دو تا وکتور که در مجموع $n + m$ پارامتر دارن رو پاس میدیم. یعنی اردر از $n * m$ به $n + m$ میرسه. ممکنه ماتریس بشکنه ممکنه نشکنه و ممکنه هم هست با کمی کاهش دقت بشکنه به دو تا وکتور ولی اردر سربار ارتباطی رو خیلی کمتر میکنه

$$A = u \cdot v^T = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \begin{bmatrix} 4 & 5 & 6 \end{bmatrix} = \begin{bmatrix} 1 \times 4 & 1 \times 5 & 1 \times 6 \\ 2 \times 4 & 2 \times 5 & 2 \times 6 \\ 3 \times 4 & 3 \times 5 & 3 \times 6 \end{bmatrix} = \begin{bmatrix} 4 & 5 & 6 \\ 8 & 10 & 12 \\ 12 & 15 & 18 \end{bmatrix}$$

یعنی فرض کنید که به نحوی بتوانیم از A به وکتورهای u و v^T برسیم (یعنی از راست به چپ برسیم)، آن وقت به جای n^2 تا پارامتر درواقع داریم $2n$ (که n اینجا ۳ است) پارامتر پاس می‌دهیم.

۴

پردازنده‌های $Intelx86$ و ARM هر دو از *Little Endian* استفاده می‌کنند. در لایه شبکه (IP) و لایه انتقال (TCP/UDP)، ترتیب بایت‌ها در هدرها *Big Endian* (*Network Byte Order*) است.

۵

توی روش BSP در هر *barrier* نودهای مختلف با هم هماهنگ میشن و همه وای میستن تا همه برسن و این روشی هست که در *map reduce* استفاده میشه و تضمین میکنه که جواب درسته. در روش SSP یک *stale threshold* داریم که هر نود به حداکثر به اون تعداد میتونه با بقیه هماهنگ نشه. اگر صفر باشه میشه BSP و اگه ۳ باشه مثلاً یعنی یه نود میتونه ۳ تا *barrier* رو بدون هماهنگی بره جلو. نکته خوبش اینه که توی قبلی ممکنه یکی کند تر باشه یه مرحله و توی مرحله بعد اونی که تند بوده کند بشه و اینجوری خیلی اتلاف داریم در صورتی که اینجا در SSP میتونیم چند مرحله بدون هماهنگی بریم جلو که به سرعت *train* اضافه میکنه ولی خب هر چقدر این *staleness* بیشتر باشه و نودهای دیر به دیر با هم هماهنگ کنن، مدل دیرتر *converge* میکنه. (البته با اینکه نمیدونیم مدل چقدر *diverge* میکنه، طبق اسلاید در نهایت تضمین میشه *convergence*)

۶

میدانیم که در توپولوژی *ring* استاندارد، سیستم زمانی سالم است که تمام n ماشین آن به درستی کار کنند. اگر حتی یکی از ماشین‌ها خراب شود، از کار می‌افتد.

احتمال خرابی ماشین i -ام: p_i

احتمال سالم بودن ماشین i -ام: $1 - p_i$

در حالت اول (بدون پشتیبان)، سیستم تنها زمانی کار می‌کند که همه ماشین‌ها سالم باشند. احتمال سالم ماندن کل سیستم برابر است با حاصل ضرب احتمال سالم بودن تک تک ماشین‌ها:

$$P_{success1} = \prod_{i=1}^n (1 - p_i)$$

بنابراین، احتمال خرابی سیستم در حالت اول برابر است با:

$$P_{fail1} = 1 - \prod_{i=1}^n (1 - p_i)$$

محاسبه احتمال خرابی با یک ماشین پشتیبان (P_{fail2}) وقتی یک ماشین پشتیبان داریم، سیستم در دو حالت همچنان به کار خود ادامه می‌دهد:

حالت اول: هیچ کدام از n ماشین اصلی خراب نشوند.

حالت دوم: دقیقاً یک ماشین از n ماشین اصلی خراب شود و ماشین پشتیبان سالم باشد و جایگزین آن شود.

فرض کنیم احتمال خرابی ماشین پشتیبان p_s باشد

احتمال حالت اول (همه سالم): $\prod_{i=1}^n (1 - p_i)$

احتمال حالت دوم (یکی خراب و پشتیبان سالم): برای هر ماشین k که خراب شود، بقیه باید سالم باشند و پشتیبان هم سالم باشد:

$$\Delta P = (1 - p_s) \sum_{k=1}^n \left(p_k \prod_{j \neq k} (1 - p_j) \right)$$

بنابراین احتمال موفقیت جدید ($P_{success2}$) مجموع این دو حالت است.

پس میتوان نتیجه گرفت که میزان کاهش احتمال خرابی به همین میزان احتمال حالت دوم خواهد بود.

در حل این بخش از [۱] استفاده کردم. توضیحات بیشتر در کامنت های لتک

مراجع

[1] “the elasticity of the ring topology,” <https://gemini.google.com/share/>.